

Extended Kalman Filter (EKF)

Contents

1	Introduction	1
2	Preliminary Concepts	2
3	Extended Kalman Filter	4
3.1	System Model	4
3.2	Extended Kalman Filter Algorithm	5
3.2.1	Prediction Step	5
3.2.2	Update Step	5
4	Nonlinear State Transition Model	6
5	State Vector Design and Measurement Strategy	7
A	Introduction to Quaternions	8
A.1	Quaternion Operations	8
A.2	Quaternions for Orientation	8

1 Introduction

In many engineering applications, we have a combination of *measurements* from sensors and a *mathematical model* of the system dynamics. Each of these sources provides valuable information about the system state—for example, the position, velocity, or orientation of a vehicle—but both are subject to uncertainty: measurements contain noise, and models are only approximations of reality.

The goal of the Extended Kalman Filter (EKF) is to *fuse these sources of information* in an optimal way to obtain the best possible estimate of the system state. Conceptually, the EKF works by:

- Using the model to **predict** the next state of the system based on the current estimate.
- Comparing this prediction with the actual **measurements** to compute a correction, called the *innovation* or *residual*.
- Updating the state estimate and its uncertainty in a way that balances the confidence in the model with the confidence in the measurements.

In essence, the EKF provides a recursive mechanism to continuously improve the state estimate, taking into account both the inherent uncertainty of the sensors and the approximations of the system model. This makes it especially useful for nonlinear systems, where simple filtering or averaging techniques would be insufficient.

This document is structured to guide the reader through both the theoretical background and the practical implementation of the EKF for the Nemesis OBC, with position, velocity, and orientation states. The contents are organized as follows:

- **Preliminary Concepts:** Key statistical notions, including random variables, Gaussian distributions, covariance, and multivariate distributions, which are fundamental for understanding uncertainty propagation in the EKF.
- **Extended Kalman Filter:** An overview of the EKF algorithm, including the system model, the prediction and update steps, and a discussion of the innovation and Kalman gain.
- **Nonlinear State Transition Model:** A detailed description of the specific nonlinear system implemented, including the state vector, measurement vector, and the equations governing the dynamics and orientation.
- **Sensor Time Synchronization:** Strategies to handle measurements arriving at different rates, including propagation of low-rate sensor data between updates.
- **Appendix: Introduction to Quaternions:** A reference for quaternion operations and their application to representing rotations, supporting the orientation modeling in the EKF.

This structure ensures a logical progression from foundational concepts, through algorithmic principles, to practical implementation and considerations, providing a comprehensive guide to applying the EKF to nonlinear navigation problems. The notation used in this document employs regular lowercase letters for scalars (x), bold lowercase letters for vectors ($\mathbf{v}$), regular uppercase letters for random variables (X), and bold uppercase letters for matrices and multivariate random variables or random vectors ($\mathbf{A}$, $\mathbf{X}$).

2 Preliminary Concepts

Before delving into the structure and operation of the Extended Kalman Filter (EKF), it is essential to recall a few foundational statistical notions. These concepts allow us to interpret sensor noise, quantify uncertainty, and ultimately understand why the EKF evolves uncertainty through its prediction and update phases. The following elements summarize the key ideas.

- **Random Variable.** A random variable is a function that assigns a real numerical value to the outcome of a stochastic phenomenon. It provides a mathematical bridge between an underlying random process and its quantitative representation, allowing uncertainty to be expressed and manipulated analytically. A random variable is characterized by its n-order moments, the first and second order moments are called *mean* and *covariance*.
- **Gaussian Random Variable (GRV).** Also known as the normal distribution, it is a continuous probability distribution whose probability density function (PDF) is given by

$$X \sim \mathcal{N}(\mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}. \quad (1)$$

Although real sensor noise rarely follows an exact Gaussian law, the assumption is generally justified by the Central Limit Theorem and is ubiquitous in navigation and control systems.

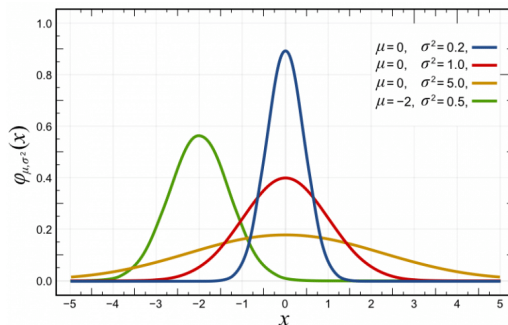


Figure 1: Normal Distribution

- **Mean.** The mean of a random variable X is defined as

$$\mu = \mathbb{E}[X], \quad (2)$$

and represents its expected or average value.

- **Variance.** The variance of X is given by

$$\text{Var}(X) = \sigma^2 = \mathbb{E}[(X - \mu)^2], \quad (3)$$

and measures the dispersion of the variable around its mean. For a Gaussian variable, approximately 68.27% of the probability mass lies within $[\mu - \sigma, \mu + \sigma]$, offering an intuitive sense of confidence in an estimate.

- **Covariance.** Given two random variables X and Y , their covariance is defined as

$$\text{Cov}(X, Y) = \sigma_{xy}^2 = \mathbb{E}[(X - \mathbb{E}(X))(Y - \mathbb{E}(Y))]. \quad (4)$$

It quantifies how the variables vary together. A positive covariance indicates that the variables tend to deviate from their means in the same direction; a negative covariance indicates the opposite.

- **Covariance Matrix.** For a set of random variables, the covariance matrix collects all variances (on the diagonal) and covariances (off-diagonal terms σ_{ij}). By definition it is symmetric, and it fully characterises the uncertainty and interdependence among the components of a multivariate estimate.
- **Independence vs. Correlation.** Two variables X and Y are uncorrelated if $\text{Cov}(X, Y) = 0$. Independence always implies zero covariance, but the converse is not necessarily true, except in the special case of jointly Gaussian variables, for which uncorrelatedness *does* imply independence.
- **Multivariate Gaussian Distribution.** A random vector $\mathbf{X} = [X_1, X_2, \dots, X_n]^T$ follows a multivariate Gaussian distribution if

$$\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{\sqrt{(2\pi)^n |\boldsymbol{\Sigma}|}} \exp \left(-\frac{1}{2} (\mathbf{X} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{X} - \boldsymbol{\mu}) \right), \quad (5)$$

where $\boldsymbol{\mu}$ is the mean vector and $\boldsymbol{\Sigma}$ the covariance matrix¹. The latter defines the shape and orientation of the uncertainty ellipsoid in n -dimensional space.

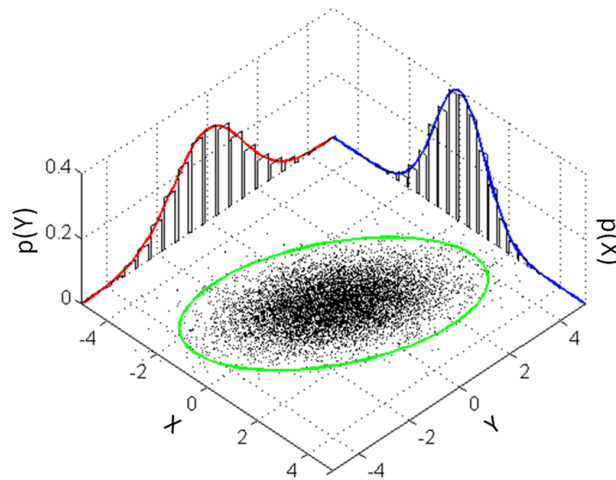


Figure 2: Multivariate Normal Distribution

¹The exponential of a matrix can be expressed analytically through its Taylor series expansion: $e^{\mathbf{A}} = \sum_{k=0}^{\infty} \frac{\mathbf{A}^k}{k!}$.

- **Linear Covariance Propagation Rule.** If a random vector $\mathbf{X}$ with covariance Σ_X is transformed linearly into

$$\mathbf{Y} = \mathbf{A} \mathbf{X} + \mathbf{b}, \quad (6)$$

then the covariance of $\mathbf{Y}$ is given by

$$\Sigma_Y = \mathbf{A} \Sigma_X \mathbf{A}^T + \Sigma_b. \quad (7)$$

where $\mathbf{b}$ is a random variable and independent from $\mathbf{X}$. This rule underpins the EKF's prediction step: even when the system is nonlinear, local linearisation allows the filter to approximate uncertainty propagation through the Jacobian of the model.

Most sensor measurements are influenced by many small, independent sources of error—for example: electrical noise, quantization, mechanical vibrations, temperature fluctuations, and minor manufacturing imperfections. Each of these error sources can be thought of as a small random variable contributing to the total measurement error.

The Central Limit Theorem (CLT) states that the sum of a large number of independent, identically distributed random variables, each with finite mean and variance, tends toward a Gaussian (normal) distribution, regardless of the individual distributions of the errors.

3 Extended Kalman Filter

3.1 System Model

Consider a nonlinear system described by:

$$\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}, \quad \mathbf{w}_{k-1} \sim \mathcal{N}(0, \mathbf{Q}) \quad (8)$$

$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(0, \mathbf{R}) \quad (9)$$

where:

- $\mathbf{x}_{k-1}$ is the state vector at time step $k - 1$.
- $\mathbf{x}_k$ is the state vector at time step k .
- $\mathbf{u}_{k-1}$ is the control input at time step $k - 1$.
- $\mathbf{z}_k$ is the measurement vector at time step k .
- $\mathbf{w}_{k-1}$ and $\mathbf{v}_k$ are process and measurement noise, modeled as zero-mean Gaussian with covariance matrices $\mathbf{Q}$ and $\mathbf{R}$, assumed diagonal (no correlation between different random variables).
- $\mathbf{f}(\cdot)$ and $\mathbf{h}(\cdot)$ are the nonlinear state transition and observation functions.

For numerical implementation in the Extended Kalman Filter, these nonlinear functions are *linearized around the current estimate* using their Jacobians, yielding the approximate linear system:

$$\hat{\mathbf{x}}_k \approx \mathbf{J}_{k-1} \mathbf{x}_{k-1} + \mathbf{B}_{k-1} \mathbf{u}_{k-1} \quad (10)$$

$$\hat{\mathbf{z}}_k \approx \mathbf{H}_k \mathbf{x}_k, \quad (11)$$

where $\mathbf{J}_{k-1}$ and $\mathbf{H}_k$ are Jacobian matrices of $\mathbf{f}$ and $\mathbf{h}$ evaluated at the current state estimate and $\mathbf{B}_{k-1}$ is the Jacobian matrix of $\mathbf{f}$ with respect to $\mathbf{u}_{k-1}$. This linearization allows the EKF to propagate the state and covariance in a computationally tractable way while still accounting for nonlinear dynamics.

3.2 Extended Kalman Filter Algorithm

The Extended Kalman Filter (EKF) is a recursive estimator designed to handle nonlinear systems. It operates in two main steps: *Prediction* and *Update*. The matrices at step k corresponding to the *prediction* phase are denoted by the subscript $\{\cdot\}_{k|k-1}$, representing the state and covariance predicted from information up to step $k-1$. After incorporating the measurement at step k , the updated estimates are denoted by $\{\cdot\}_{k|k}$, representing the corrected state and covariance that combine both the model prediction and the measurement information. This notation allows a clear distinction between the *prior* (predicted) and *posterior* (updated) estimates within the recursive filtering process.

3.2.1 Prediction Step

In the **prediction step**, the filter uses the system model to propagate the previous state estimate forward in time, producing the predicted state $\hat{\mathbf{x}}_{k|k-1}$. The state transition function $\mathbf{f}$ captures the system dynamics, while the process noise covariance $\mathbf{Q}$ accounts for uncertainty in the model. The linearization of $\mathbf{f}$ around the current estimate, represented by the Jacobian $\mathbf{F}_k$, allows the filter to propagate the state covariance $\mathbf{P}_{k|k-1}$, which quantifies the uncertainty of the predicted state.

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{f}(\hat{\mathbf{x}}_{k-1|k-1}) \quad (12)$$

$$\mathbf{F}_k = \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1|k-1}} \quad (13)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q} \quad (14)$$

Brief derivation:

$$\begin{aligned} \hat{\mathbf{x}}_{k|k-1} &= \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{w}_{k-1} \\ \mathbf{P}_{k|k-1} &= \text{Cov}(\hat{\mathbf{x}}_{k|k-1}) \\ &= \text{Cov}(\mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{w}_{k-1}) \\ &= \text{Cov}(\mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1}) + \mathbf{Q} \\ &= \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q} \end{aligned}$$

where in the last step, the *linear covariance propagation rule* has been applied.

3.2.2 Update Step

During the **update step**, a new measurement $\mathbf{z}_k$ is incorporated to correct the prediction. The measurement function $\mathbf{h}$ maps the state to the measurement space, and its Jacobian $\mathbf{H}_k$ linearizes this mapping. The innovation covariance $\mathbf{S}_k$ represents the expected uncertainty of the predicted measurement. Using the Kalman gain $\mathbf{G}_k$, the filter balances the predicted state and the measurement, updating both the state estimate $\hat{\mathbf{x}}_{k|k}$ and its covariance $\mathbf{P}_{k|k}$. This update reduces the uncertainty by incorporating the information from the new measurement, resulting in an improved estimate of the system state.

$$\mathbf{H}_k = \left. \frac{\partial \mathbf{h}}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k|k-1}} \quad (15)$$

$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R} \quad (16)$$

$$\mathbf{G}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1} \quad (17)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{G}_k (\mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_{k|k-1})) \quad (18)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{G}_k \mathbf{H}_k) \mathbf{P}_{k|k-1}, \quad (19)$$

where,

- $\mathbf{H}_k$: Jacobian of the measurement function $\mathbf{h}(\cdot)$, linearized around the predicted state $\hat{\mathbf{x}}_{k|k-1}$. It maps small changes in the state to changes in the measurement.
- $\mathbf{S}_k$: Innovation (or residual) covariance. It represents the expected uncertainty of the predicted measurement and combines both the predicted state uncertainty and the measurement noise $\mathbf{R}$.
- $\mathbf{G}_k$: Kalman gain. It determines the relative weight of the measurement versus the prediction in updating the state estimate. It is derived such that the covariance of the update is minimum.
- $\hat{\mathbf{x}}_{k|k}$: Updated state estimate. This is the corrected state after incorporating the measurement, weighted by the Kalman gain.
- $\mathbf{P}_{k|k}$: Updated covariance. It reflects the reduced uncertainty of the state estimate after including the measurement information.

Note on the innovation (residual): The term

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_{k|k-1})$$

$$\text{Cov}(\mathbf{y}_k) = \mathbf{S}_k$$

represents the innovation, or residual, which is the difference between the actual measurement and the predicted measurement. It captures the new information provided by the measurement, and its covariance $\mathbf{S}_k$ quantifies the expected uncertainty in this difference, combining both the predicted state uncertainty and the measurement noise.

Note on the computation of $\mathbf{G}_k$: The matrix $\mathbf{G}_k$, also known as the Kalman gain, is chosen to minimise the posterior covariance $\mathbf{P}_{k|k}$. Formally, by taking the covariance of the update equation

$$\mathbf{P}_{k|k} = \text{Cov}(\hat{\mathbf{x}}_{k|k}) = \text{Cov}[\hat{\mathbf{x}}_{k|k-1} + \mathbf{G}_k(\mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_{k|k-1}))], \quad (20)$$

differentiating with respect to $\mathbf{G}_k$, and solving for the minimiser, one obtains

$$\mathbf{G}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \mathbf{S}_k^{-1}, \quad \mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}. \quad (21)$$

This choice ensures that the update step incorporates the measurement optimally, reducing uncertainty in the estimated state.

4 Nonlinear State Transition Model

This section presents the dynamic model employed in the Extended Kalman Filter. The state vector is defined as

$$\mathbf{x} = [h, v_z, q_w, q_x, q_y, q_z]^T, \quad (22)$$

which comprises the altitude, vertical velocity, and attitude expressed as a unit quaternion. The measurement vector is event-driven and adapts to whichever sensor provides the most recent update:

$$\mathbf{z} = \begin{cases} [z_{\text{GPS}}] & \text{upon reception of a new GPS measurement,} \\ [z_{\text{BARO}}] & \text{upon reception of a new barometric reading.} \end{cases} \quad (23)$$

The input vector is obtained directly from the inertial measurement unit (IMU):

$$\mathbf{u} = [a_x^b, a_y^b, a_z^b, g_x^b, g_y^b, g_z^b]^T, \quad (24)$$

where the superscript b denotes quantities expressed in the body frame. Translational dynamics are expressed in the inertial frame, whereas attitude evolves according to quaternion kinematics.

The predicted altitude and vertical velocity for the next time step are given by

$$h_k = h_{k-1} + v_{z,k-1} \Delta t + \frac{1}{2} a_z^I \Delta t^2, \quad (25)$$

$$v_{z,k} = v_{z,k-1} + a_z^I \Delta t, \quad (26)$$

where a_z^I is the acceleration expressed in the inertial frame. It is computed as

$$a_z^I = \mathbf{k} \cdot (\mathbf{q} \otimes \mathbf{a}^b \otimes \mathbf{q}^*), \quad (27)$$

i.e., the projection of the body-frame acceleration onto the inertial vertical axis.

Finally, the quaternion time evolution obeys

$$\mathbf{q}_k = \mathbf{q}_{k-1} + \frac{1}{2} \begin{bmatrix} 0 & -g_x^b & -g_y^b & -g_z^b \\ g_x^b & 0 & g_z^b & -g_y^b \\ g_y^b & -g_z^b & 0 & g_x^b \\ g_z^b & g_y^b & -g_x^b & 0 \end{bmatrix} \mathbf{q}_{k-1} \Delta t. \quad (28)$$

Finding an analytical expression for $\mathbf{J}$ and $\mathbf{B}$ becomes extremely tedious, thus, it will be computed numerically, avoiding human mistakes in the computation of the analytical expression.

5 State Vector Design and Measurement Strategy

The initial implementation employed a 6-dimensional state vector

$$\mathbf{x} = [z, v_z, q_w, q_x, q_y, q_z]^T, \quad (29)$$

tracking only the vertical position and velocity along with the orientation quaternion. The measurement vector was fixed at 8 dimensions,

$$\mathbf{z} = [a_x, a_y, a_z, \omega_x, \omega_y, \omega_z, z_{\text{GPS}}, z_{\text{baro}}]^T, \quad (30)$$

including all sensor readings simultaneously. While straightforward, this design exhibited several drawbacks: it neglected horizontal position information from the GPS, treated IMU readings as measurements rather than as control inputs driving the system dynamics, and imposed the need for all sensors to be synchronized into a single measurement vector, increasing implementation complexity and computational load.

To address these limitations, a revised approach is proposed while maintaining the same 6-dimensional state vector. The key modification is to render the measurement vector dynamic, incorporating only the data available at each time step. Specifically, when GPS data is received, the measurement vector consists solely of the three position components,

$$\mathbf{z}_{\text{GPS}} = [z_{\text{GPS}}], \quad (31)$$

whereas a barometer reading contributes only the altitude measurement,

$$\mathbf{z}_{\text{baro}} = [z_{\text{baro}}]. \quad (32)$$

The IMU data is removed entirely from the measurement vector and instead used as control input in the prediction step, which runs at the full IMU frequency. This asynchronous strategy simplifies the implementation, reduces computational cost—since the innovation covariance to be inverted is now a scalar rather than 8×8 matrix—and avoids the need for placeholder values or sensor synchronization.

For future implementations requiring higher precision, it is recommended to adopt a full 10-dimensional state vector,

$$\mathbf{x} = [p_x, p_y, p_z, v_x, v_y, v_z, q_w, q_x, q_y, q_z]^T, \quad (33)$$

which allows the filter to properly integrate IMU accelerations in all directions, utilize the complete GPS position information, and maintain physically consistent dynamics where sensor measurements align with the estimated states. In this configuration, the dynamic, single-component measurement vector described above remains applicable, offering the same computational and estimation advantages while providing enhanced accuracy due to the richer state representation. Then the measurement vector would include all the components provided by the GPS.

A Introduction to Quaternions

Quaternions provide a compact and efficient way to represent the orientation of an object in three-dimensional space, avoiding some of the limitations of Euler angles, such as gimbal lock. A quaternion is a four-dimensional number, typically written as:

$$\mathbf{q} = q_w + q_x \mathbf{i} + q_y \mathbf{j} + q_z \mathbf{k} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}, \quad (34)$$

where q_w is the scalar part and (q_x, q_y, q_z) form the vector part.

A.1 Quaternion Operations

The main operations used in orientation representation include:

- **Conjugate:** $\mathbf{q}^* = q_w - q_x \mathbf{i} - q_y \mathbf{j} - q_z \mathbf{k}$, which represents the inverse rotation for unit quaternions.
- **Norm:** $\|\mathbf{q}\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2}$; unit quaternions have norm 1.
- **Multiplication:** Given two quaternions $\mathbf{q}_1 = [q_1 \ \mathbf{v}_1]^T$ and $\mathbf{q}_2 = [q_2 \ \mathbf{v}_2]^T$, their product $\mathbf{q}_1 \otimes \mathbf{q}_2$ combines the rotations represented by each quaternion.

$$\mathbf{q}_1 \otimes \mathbf{q}_2 = \begin{bmatrix} q_1 q_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \\ q_1 \mathbf{v}_2 + q_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} q_1 q_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \\ q_1 q_2 + q_1 q_2 + q_1 q_2 - q_1 q_2 \\ q_1 q_2 - q_1 q_2 + q_1 q_2 + q_1 q_2 \\ q_1 q_2 + q_1 q_2 - q_1 q_2 + q_1 q_2 \end{bmatrix}$$

$$\mathbf{q}_1 \otimes \mathbf{q}_2 = \begin{bmatrix} s_1 \\ \mathbf{v}_1 \end{bmatrix} \otimes \begin{bmatrix} s_2 \\ \mathbf{v}_2 \end{bmatrix} = \begin{bmatrix} s_1 s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2 \\ s_1 \mathbf{v}_2 + s_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2 \end{bmatrix}. \quad (35)$$

- **Inverse:** For a unit quaternion, the inverse is equal to its conjugate: $\mathbf{q}^{-x} = \mathbf{q}^*$.

A.2 Quaternions for Orientation

To represent orientation:

- A unit quaternion corresponds to a rotation around a unit axis $\mathbf{u}$ by an angle θ :

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) \\ \mathbf{u} \sin(\theta/2) \end{bmatrix}. \quad (36)$$

- Rotating a vector $\mathbf{v}$ in 3D space is done via quaternion multiplication:

$$\mathbf{v}' = \mathbf{q} \otimes \mathbf{v} \otimes \mathbf{q}^* = \mathbf{R}(\mathbf{q})\mathbf{v}, \quad (37)$$

where $\mathbf{v}$ is represented as a pure quaternion with zero scalar part.

- Quaternions avoid singularities and allow smooth interpolation (slerp) between orientations, making them ideal for aerospace, robotics, and computer graphics.

The rotation matrix built from the quaternion components yields:

$$\mathbf{R}(\mathbf{q}) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}.$$